

Tích hợp Partner ↔ P2P: Hướng dẫn Toàn diện

Tài liệu này cung cấp hướng dẫn chi tiết về cách tích hợp nền tảng P2P của chúng tôi với hệ thống của đối tác, tập trung vào các khía cạnh kỹ thuật và sản phẩm. Mục tiêu là cho phép người dùng từ sàn đối tác sử dụng P2P một cách liền mạch, duy trì số dư ví trên sàn đối tác mà không cần đăng nhập lại.

Giá trị Cốt lõi và Phạm vi Tích hợp

Truy cập Một chạm (One-click Access)

Người dùng sàn đối tác có thể truy cập P2P mà không cần đăng nhập lại, nhờ vào cơ chế Đăng nhập một lần (SSO) an toàn và hiệu quả.

Duy trì Số dư tại Đối tác

Số dư tài sản của người dùng vẫn được giữ nguyên trên sàn đối tác. P2P chỉ thực hiện các lệnh khóa/mở/chuyển thông qua Funds API.

Hỗ trợ Giao dịch Toàn diện

Hệ thống hỗ trợ các thao tác khóa (lock), mở khóa (unlock) và chuyển (transfer) tài sản dựa trên từng giao dịch (order/trade), đảm bảo tính bất biến (idempotent) theo requestId.

Việc tích hợp này mang lại trải nghiệm người dùng liền mạch và an toàn, đồng thời tối ưu hóa quá trình quản lý tài sản giữa các nền tảng. Nó cho phép đối tác mở rộng dịch vụ P2P cho người dùng hiện tại mà không làm thay đổi thói quen quản lý tài sản của họ.

Kiến trúc Logic Hệ thống



Kiến trúc logic của hệ thống được thiết kế để đảm bảo sự tương tác hiệu quả và bảo mật giữa P2P và nền tảng của đối tác. Trình duyệt đóng vai trò là điểm khởi đầu cho người dùng, chuyển hướng họ đến giao diện P2P thông qua một mã một lần duy nhất.

- **Browser:** Chuyển hướng người dùng sang P2P bằng một one-time code đã được ký.
- **P2P Backend (9000):** Xác thực yêu cầu thông qua HMAC, ánh xạ người dùng (map user), và gọi Partner Funds API khi người dùng thuộc đối tác.
- **Partner Backend:** Cung cấp cơ chế SSO Issue và các API quản lý quỹ (Funds API) như kiểm tra số dư (balances), khóa (lock), mở khóa (unlock), và chuyển (transfer) tài sản.
- **Khoá dữ liệu:** Redis được sử dụng để lưu trữ nonce và code tạm thời. Postgres phục vụ lưu trữ log lệnh (order/trade) của P2P, trong khi đối tác quản lý số dư trong cơ sở dữ liệu riêng của họ. WebSocket (9002) được dùng cho các tương tác thời gian thực.

Yêu cầu Chuẩn bị từ phía Đối tác

Giao diện Người dùng (UI)

Cần bổ sung nút hoặc liên kết "P2P" trên giao diện web/ứng dụng của đối tác. Khi người dùng nhấp vào, họ sẽ được chuyển hướng theo luồng SSO đã định.

Backend SSO Caller

Thiết lập một endpoint backend để thực hiện ký HMAC với partnerId và sharedSecret, sau đó gọi POST /api/sso/issue của P2P.

Funds API (Server-to-Server)

Triển khai các API sau để P2P tương tác: balances, lock, unlock, và transfer. Các API này phải trả về số thập phân dưới dạng chuỗi và đảm bảo tính bất biến (idempotent) theo requestId.

Bảo mật & Giám sát

Đảm bảo lưu trữ sharedSecret an toàn. Thực hiện kiểm tra timestamp $\pm$ skew, chống sử dụng lại nonce (replay attack), và duy trì log/audit đầy đủ cho các giao dịch quan trọng.

Yêu cầu Chuẩn bị từ phía P2P

Cấp thông tin cho Đối tác

- Cung cấp **partnerId** và **sharedSecret** sau khi tạo qua POST /api/integration/partners.
- Cung cấp **clientId/clientSecret** cho Funds API của đối tác.
- Thông báo **base URL** của P2P và **UI URL** để đối tác có thể redirect người dùng.



Cấu hình Môi trường P2P

- Thiết lập biến môi trường như `APP_SSO_PARTNERS=SAN_A:<secret>`.
- Cấu hình `APP_PARTNER_API_PARTNERS=SAN_A=http://partner_mock:9001` và `APP_PARTNER_API_CLIENT_SECRET=<...>`.
- Đảm bảo các dịch vụ Redis, RabbitMQ, Postgres được cấu hình đúng theo docker-compose.

Giám sát Hệ thống

Thiết lập cảnh báo khi Funds API của đối tác trả về lỗi (ví dụ: 401/409/timeout) và theo dõi chặt chẽ độ trễ (latency) cũng như tỷ lệ lỗi để đảm bảo hoạt động ổn định.

Luồng Đăng nhập một lần (SSO)



1. Người dùng click P2P

Người dùng đã đăng nhập trên sàn đối tác và quyết định truy cập P2P.



2. Partner Backend gọi SSO

Backend của đối tác ký HMAC và gọi POST /api/sso/issue với thông tin người dùng.



3. Chuyển hướng Trình duyệt

P2P nhận {code, expiresIn} và chuyển hướng trình duyệt (302 Redirect) đến <https://p2p.../sso?code=<code>>.



4. P2P UI trao đổi Code

Giao diện P2P gọi POST /api/sso/exchange để đổi code lấy JWT {token, userId}.

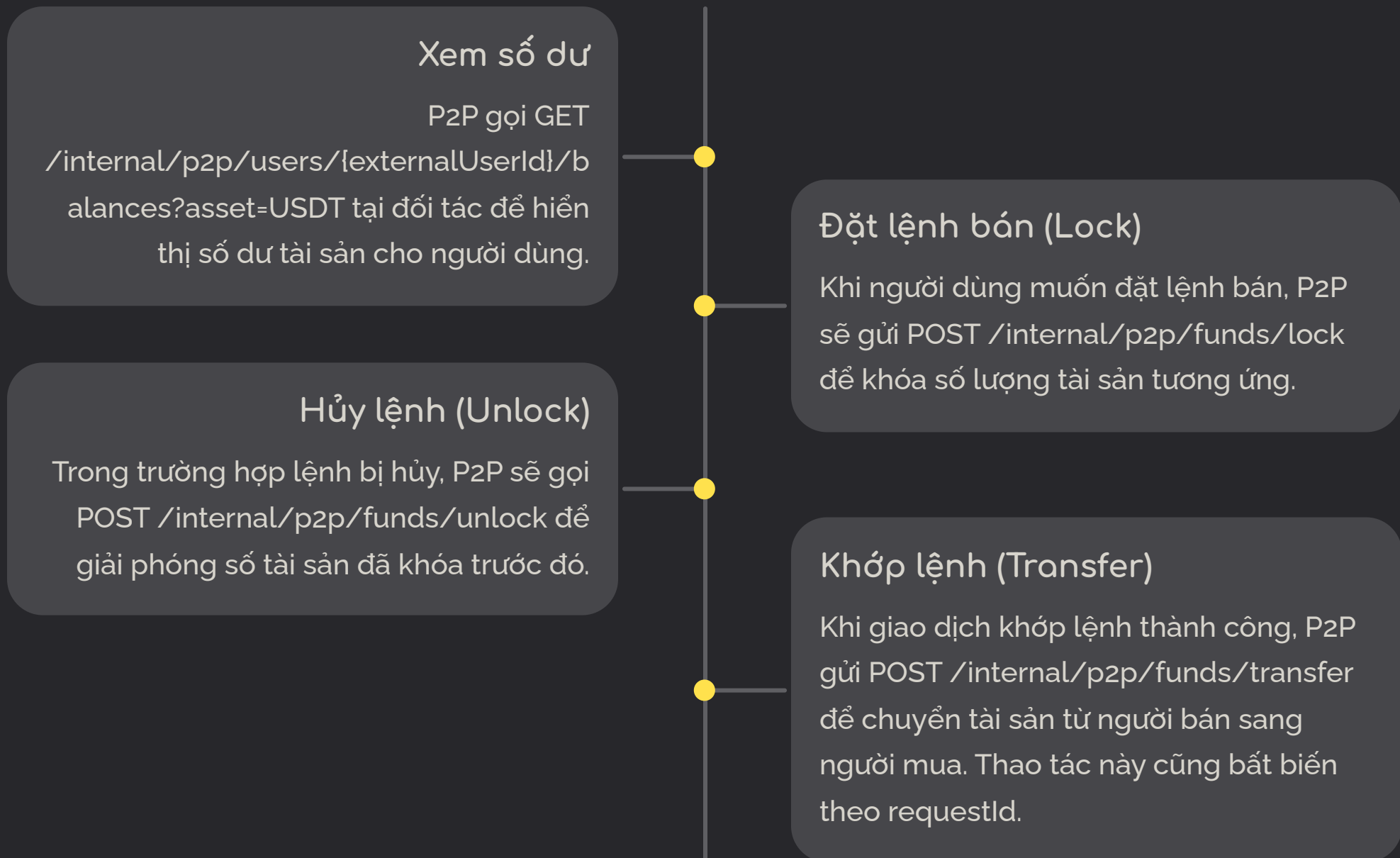


5. Giao tiếp qua JWT

Từ lúc này, tất cả các yêu cầu API P2P đều sử dụng Authorization: Bearer <token>.

Luồng SSO này đảm bảo rằng người dùng có thể chuyển đổi giữa nền tảng đối tác và P2P một cách an toàn và tiện lợi, giảm thiểu rào cản đăng nhập.

Luồng Quản lý Ví và Giao dịch



Các thao tác này đảm bảo tính toàn vẹn của số dư và giao dịch, với mỗi bước đều được thực hiện thông qua API của đối tác, duy trì quyền kiểm soát tài sản tại sàn của họ.

Tổng quan về HMAC: Bảo mật Giao tiếp

Cấu trúc Headers

Mỗi yêu cầu API được bảo vệ bởi HMAC đều bao gồm các headers sau:

- **X-Partner-Id** (cho /api/sso/issue) hoặc **X-P2P-Id** (cho funds API).
- **X-Timestamp**: Thời gian gửi yêu cầu.
- **X-Nonce**: Một chuỗi ngẫu nhiên duy nhất cho mỗi yêu cầu.
- **X-Signature**: Chữ ký HMAC của yêu cầu.



Chuỗi Ký và Xác thực

Chuỗi dùng để ký được xây dựng từ các thành phần sau:

```
METHOD + "\n" + PATH + "\n" + timestamp + "\n" + nonce + "\n" + sha256Hex(body)
```

Để xác thực yêu cầu, hệ thống sẽ kiểm tra:

- Độ lệch thời gian **timestamp** không vượt quá **APP_SSO_TIMESTAMP_SKEW_SECONDS**.
- **Nonce** chưa từng được sử dụng trước đó (chống replay attack).
- **Chữ ký** hợp lệ, được tính toán lại và so sánh với **X-Signature**.

Cơ chế HMAC đảm bảo tính toàn vẹn và xác thực của mọi yêu cầu giữa P2P và đối tác, ngăn chặn các cuộc tấn công giả mạo và thay đổi dữ liệu.

Điểm cuối API (Endpoints) Chính

P2P nhận từ Đối tác

- **POST /api/sso/issue:** Đối tác gọi để lấy mã SSO cho người dùng.
- **POST /api/sso/exchange:** UI của P2P gọi để đổi mã SSO lấy JWT.
- **POST /api/integration/partners:** (Admin) Tạo credentials cho đối tác.



P2P gọi đến Đối tác (Funds API)

- **GET /internal/p2p/users/{externalUserId}/balances:** Truy vấn số dư tài sản của người dùng.
- **POST /internal/p2p/funds/lock:** Yêu cầu khóa số dư cho giao dịch.
- **POST /internal/p2p/funds/unlock:** Yêu cầu mở khóa số dư đã bị khóa.
- **POST /internal/p2p/funds/transfer:** Yêu cầu chuyển tài sản giữa các ví.



Việc phân định rõ ràng các endpoint này giúp cả hai bên dễ dàng hiểu và triển khai các luồng tích hợp, đảm bảo sự tương thích và ổn định của hệ thống.

Rủi ro Tiềm ẩn và Lưu ý Quan trọng



Sai lệch HMAC/Thời gian

Lỗi HMAC hoặc lệch múi giờ (clock skew) có thể dẫn đến lỗi 401 (Unauthorized). Cần đồng bộ thời gian (NTP) và cấu hình `APP_SSO_TIMESTAMP_SKEW_SECONDS` phù hợp.



Thiếu Idempotency

Việc thiếu kiểm tra `requestId` cho tính bất biến (idempotency) có thể gây ra các lỗi nghiêm trọng như khóa hoặc chuyển tiền hai lần (double lock/transfer).



Timeout Funds API

Các lỗi timeout từ Funds API của đối tác cần được xử lý bằng cơ chế thử lại (retry) với khoảng thời gian tăng dần (backoff) và đảm bảo tính bất biến.



Vấn đề CORS/UI

Khi triển khai thực tế, cần đảm bảo cấu hình `whitelists origin` cho CORS một cách chính xác để tránh các lỗi bảo mật và hiển thị giao diện.

Việc nhận diện và chủ động quản lý các rủi ro này là chìa khóa để đảm bảo quá trình tích hợp diễn ra suôn sẻ, hệ thống hoạt động ổn định và an toàn.